

Lekce 2: Základy JavaScriptu pro automatizaci testování

1. Osnova obsahu

A. Úvod do JavaScriptu

- **Co je JavaScript?**
 - Vysoce úroňový, interpretovaný programovací jazyk.
 - Primárně se používá pro přidání interaktivity na webové stránky.
- **JavaScript v automatizaci testování:**
 - Důležitost znalosti JavaScriptu pro psaní testů v Cypressu.
 - Jak JavaScript spolupracuje s Cypress pro plynulé skriptování testů.

Co je JavaScript?

Definice:

JavaScript je vysoce úroňový, interpretovaný programovací jazyk, který se primárně používá pro vytváření a správu dynamického obsahu webových stránek, umožňující interaktivní prvky na webech.

Podrobné vysvětlení:

- **Role ve vývoji webu:**
 - JavaScript je klíčový pro přidání interaktivity na webové stránky. Umožňuje vývojářům vytvářet prvky jako posuvníky, formuláře, animace a interaktivní mapy.
- **Spuštění na straně klienta:**
 - Obvykle běží v prohlížeči uživatele, což umožňuje interakci v reálném čase bez nutnosti neustálé komunikace se serverem.
- **Všestrannost:**
 - Mimo prohlížeč se JavaScript používá i na straně serveru v prostředí jako je Node.js, což umožňuje vývoj full-stack aplikací v jednom jazyce.
- **Integrace s HTML a CSS:**
 - Pracuje spolu s HTML (struktura) a CSS (vzhled) pro vytváření kompletních, interaktivních webových aplikací.

Proč je JavaScript nejmilovanější i nenáviděnější jazyk?

Definice:

JavaScript je známý pro svou všestrannost a všudypřítomnost, díky čemuž je oblíbený mezi vývojáři.

Jeho flexibilita a jisté zvláštnosti však mohou také způsobovat frustraci, což mu v komunitě vysloužilo jak lásku, tak nenávist.

Podrobné vysvětlení:

- **Proč je nejoblíbenější:**

- **Všudypřítomnost:** JavaScript je základním kamenem webu, běží téměř na každé webové stránce.
- **Všestrannost:** Lze použít pro front-end i back-end vývoj.
- **Bohatý ekosystém:** Rozsáhlé knihovny a frameworky (např. React, Angular, Vue.js) zvyšují produktivitu.
- **Aktivní komunita:** Velká, podporující komunita přispívá k neustálému vylepšování a zdrojům.
- **Interaktivita v reálném čase:** Umožňuje vývojářům tvořit dynamická a responzivní uživatelská rozhraní.

- **Proč je nejnenáviděnější:**

- **Nekonzistence:** Zvláštnosti jako přetypování a nečekané chování mohou vést k chybám a zmatku.
- **Volné typování:** Dynamické typování může ztížit správu a ladění rozsáhlého kódu.
- **Rozdíly mezi prohlížeči:** Různá interpretace JavaScriptu napříč prohlížeči může ztěžovat vývoj.
- **Rychlý vývoj:** Časté aktualizace a změny mohou být pro vývojáře obtížné sledovat.
- **Asynchronní složitost:** Správa asynchronních operací může být složitá, obzvláště pro začátečníky.

Co je vysoce úroňový programovací jazyk?

Definice:

Vysoce úroňový programovací jazyk abstrahuje od většiny detailů hardwaru, což umožňuje vývojářům psát programy pomocí lidsky čitelné syntaxe a konceptů, místo přímého zacházení se strojovým kódem.

Podrobné vysvětlení:

- **Abstrakce:**

- Vysoké jazyky nabízejí abstrakce jako proměnné, cykly a funkce, což usnadňuje psaní komplexních programů bez starosti o nízkoúroňové operace.

- **Snadnost použití:**

- Jsou navrženy tak, aby byly snadné na čtení a psaní, což umožňuje zaměřit se na řešení problémů, nikoli na hardwarové detaily.

- **Přenositelnost:**
 - Programy napsané ve vysokých jazycích často běží na různém hardwaru s malými nebo žádnými úpravami.
- **Příklady:**
 - JavaScript, Python, Java, C# a Ruby jsou všechny vysoce úrovněvé jazyky.
- **Srovnání s nízkoúrovňovými jazyky:**
 - Oproti nízkoúrovňovým jazykům (např. assembler) automaticky spravují paměť, I/O a další systémové úkoly.

Proč je JavaScript vhodný pro psaní testů?

Definice:

JavaScript je dobře vhodný pro psaní testů, zejména webových aplikací, díky nativní integraci s prohlížeči, asynchronním možnostem a bohatému ekosystému testovacích frameworků jako Cypress.

Podrobné vysvětlení:

- **Nativní integrace s prohlížečem:**
 - Protože JavaScript běží v prohlížeči, může přímo interagovat s prvky stránky, což je ideální pro end-to-end testování.
- **Asynchronní zpracování:**
 - Schopnost JavaScriptu spravovat asynchronní operace umožňuje testovat dynamické webové aplikace, které závisí na asynchronním získávání dat a uživatelských interakcích.
- **Bohatý ekosystém:**
 - Nástroje jako Cypress, Jest nebo Mocha poskytují robustní možnosti pro psaní, spouštění a správu testů.
- **Znovupoužitelnost:**
 - Běžné patterny a utility lze znovu použít ve více testovacích sadách, což zvyšuje efektivitu.
- **Komunita a podpora:**
 - Velká komunita zajišťuje neustálý vývoj, dostatek zdrojů a podporu pro testování a nástroje.
- **Flexibilita:**
 - JavaScript umožňuje jednotkové i integrační testy v jednom jazyce.

Proměnné a datové typy

- **Proměnné:**
 - Deklarace pomocí `var`, `let` a `const`.
 - Rozdíly ve scope mezi `var`, `let` a `const`.
 - Nejlepší praktiky pojmenovávání proměnných.
- **Datové typy:**

- Primitivní typy: `String` , `Number` , `Boolean` , `Null` , `Undefined` , `Symbol` .
- Složené typy: `Object` , `Array` .
- Porozumění přetypování a kontrole typů.

Jaké jsou rozdíly mezi `var` , `let` a `const` ?**

Definice:

`var` , `let` a `const` jsou klíčová slova v JavaScriptu používaná k deklaraci proměnných, každé s jiným scope a vlastnostmi změnitelnosti.

Podrobné vysvětlení:

- **var :**
 - **Funkční scope:** Proměnné deklarované pomocí `var` platí v rámci nejbližší funkce.
 - **Hoisting:** `var` je vynášen na začátek scope, tj. lze k němu přistupovat před deklarací (bude však `undefined` do přiřazení hodnoty).
 - **Znovudeklarace a přiřazení:** Může být znovu deklarováno a přeloženo ve stejném scope.
- **let :**
 - **Blokový scope:** `let` platí v rámci nejbližšího bloku (`{ }`), např. uvnitř cyklů nebo podmínek.
 - **Bez hoistingu:** Přestože je hoistováno, není inicializováno do přiřazení, což vytváří „temporální mrtvou zónu“.
 - **Přiřazení:** Může být přeloženo, ale ne znovu deklarováno ve stejném scope.
- **const :**
 - **Blokový scope:** Stejně jako `let` , je scope na blok.
 - **Bez hoistingu:** Hoistováno, ale ne inicializováno do přiřazení.
 - **Nelze přeložit:** Proměnnou `const` nelze přeložit po první deklaraci. Pokud je objekt/array, jeho vlastnosti/prvky lze měnit.
 - **Imutabilita:** Podporuje neměnné vazby, což vede k bezpečnějšímu a předvídatelnějšímu kódu.

Co jsou primitivní datové typy?

Definice:

Primitivní typy jsou nejzákladnější datové typy v JavaScriptu, které představují jednotlivé, neměnné hodnoty. Nejsou to objekty a nemají metody.

Podrobné vysvětlení:

- **Seznam primitivních typů:**
 - **String:** Textová data (např. `"Hello, World!"`).
 - **Number:** Celá a desetinná čísla (např. `42` , `3.14`).

- **Boolean:** Logické hodnoty (`true` nebo `false`).
- **Undefined:** Proměnná byla deklarována, ale nebyla jí přiřazena hodnota.
- **Null:** Úmyslné neexistování objektové hodnoty.
- **Symbol:** Unikátní a neměnné identifikátory (představeno v ES6).
- **BigInt:** Celá čísla s libovolnou přesností (představeno v ES2020).
- **Charakteristiky primitivních typů:**
 - **Neměnné:** Po vytvoření hodnotu nelze změnit. Operace vytvoří novou hodnotu.
 - **Závislé na hodnotě:** Uchovávají skutečnou hodnotu, ne odkaz na objekt.
- **Příklady:**

```
let name = "Alice"; // String
let age = 30; // Number
let isStudent = true; // Boolean
let unassigned; // Undefined
let emptyValue = null; // Null
let uniqueId = Symbol('id'); // Symbol
let largeNumber = 9007199254740991n; // BigInt
```

• Konvence pojmenovávání

i. Používejte popisná a smysluplná jména:

- **Srozumitelnost:** Vyberte jména, která jasně popisují účel nebo hodnotu proměnné.

```
let totalPrice = 100; // Jasný účel
let tp = 100; // Nejasné
```

- **Vyhýbejte se nejednoznačností:** Nepoužívejte obecná jména jako `data` nebo `value` , pokud není jejich význam jasný.

```
let userData = { name: "Alice", age: 25 }; // Jasně
let data = { name: "Alice", age: 25 }; // Nejasné
```

ii. Dodržujte konzistentní konvence pojmenování:

- **camelCase:** Začněte malým písmenem, další slova začínají velkým.

```
let firstName = "John";
let totalAmount = 250;
```

- **Vyhýbejte se snake_case a PascalCase:** Pro proměnné a funkce používejte camelCase.

```
// Doporučeno
let userAge = 30;
```

```
// Nedoporučeno
let user_age = 30;
let UserAge = 30;
```

iii. Používejte vyslovitelná jména:

- o **Snadná komunikace:** Jména by měla být lehce vyslovitelná pro diskusi s týmem.

```
let userEmail = "user@example.com"; // Snadné vyslovit
let u_e = "user@example.com"; // Těžce vyslovitelné
```

iv. Vyhýbejte se zkratkám a akronymům:

- o **Plná slova:** Pro jasnost používejte celá slova, pokud zkratka není všeobecně známá.

```
let maxHeight = 200; // Jasné
let mh = 200; // Nejasné

let url = "https://example.com"; // Jasné
let u = "https://example.com"; // Nejasné
```

v. Používejte jednotná čísla pro jednotlivé položky a množná pro kolekce:

- o **Konzistence u kolekcí:**

```
let user = { name: "Alice" }; // Jednotlivá položka
let users = [{ name: "Alice" }, { name: "Bob" }]; // Kolekce
```

vi. Nepoužívejte rezervovaná slova a speciální znaky:

- o **JavaScript rezervovaná slova:** Nepoužívejte klíčová slova jako `class`, `return`, `var` atd. jako názvy proměnných.

```
// Nevhodné
let class = "Mathematics"; // SyntaxError

// Použijte raději
let className = "Mathematics";
```

- o **Žádné speciální znaky:** Používejte pouze písmena, čísla a podtržítka (`_`). Vyhněte se mezerám a speciálním znakům.

```
// Platné
let firstName = "John";

// Neplatné
let first-name = "John"; // SyntaxError
```

vii. **Uved'te účel proměnné pomocí kontextu:**

- **Kontextové nápovědy:** Používejte prefixy/suffixy indikující roli nebo typ proměnné.

```
let isLoggedIn = true; // Boolean flag
let userCount = 50; // Číselný počet
let userList = ["Alice", "Bob"]; // Pole
```

viii. **Používejte krátká, ale smysluplná jména:**

- **Rovnováha mezi délkou a srozumitelností:** Jména by měla být popisná, ne ale příliš dlouhá.

```
let userProfile = { name: "Alice", age: 25 }; // Dobré
let userProfileInformationDetails = { name: "Alice", age: 25 }; // Příliš dlouhé
```

ix. **Dodržujte konzistenci u podobných proměnných:**

- **Jednotnost:** Podobné proměnné pojmenovávejte jednotným způsobem napříč kódem.

```
let userName = "Alice";
let adminName = "Bob";
// Obě mají příponu 'Name'
```

x. **Používejte čísla v názvech jen když je to smysluplné:**

- **Smysluplné použití:** Čísla v názvu užívejte jen pokud zvyšují srozumitelnost.

```
let user2 = "Charlie"; // Raději se vyhnout
let alternateUser = "Charlie"; // Lepší
```

Co je typeof ?

Definice:

typeof je operátor v JavaScriptu, který vrátí řetězec určující typ zadaného operandu, což pomáhá vývojářům identifikovat datové typy při ladění a validaci.

Podrobné vysvětlení:

- **Základní syntaxe:**

`typeof` operand

- **Operand:** Proměnná nebo hodnota, jejíž typ chcete zjistit.

- **Běžné použití:**

```
typeof "Hello"; // Vrací: "string"
typeof 42; // Vrací: "number"
typeof true; // Vrací: "boolean"
typeof undefined; // Vrací: "undefined"
typeof null; // Vrací: "object" (známá zvláštnost JS)
typeof { name: "Alice" }; // Vrací: "object"
typeof [1, 2, 3]; // Vrací: "object"
typeof function() {}; // Vrací: "function"
typeof Symbol('id'); // Vrací: "symbol"
typeof 10n; // Vrací: "bigint"
```

- **Praktické aplikace:**

- **Kontrola typů:**

- Ověření, že proměnné mají očekávané datové typy před provedením operací.

```
if (typeof score === "number") {
  console.log("Score je číslo");
} else {
  console.log("Score není číslo");
}
```

- **Validace parametrů funkcí:**

- Kontrola typů argumentů předávaných funkcím.

```
function greet(name) {
  if (typeof name !== "string") {
    console.error("Jméno musí být string");
    return;
  }
  console.log("Ahoj, " + name + "!");
}
greet("Bob"); // Validní
greet(123); // Zobrazí chybu
```

- **Omezení a poznámky:**

- **null vrací "object":**

- I když je null primitivní typ, `typeof null` vrací "object", což může mást.
- **Pole a objekty:**
 - Pole i objekty vrací "object". K rozlišení použijte např. `Array.isArray()`.

```
typeof []; // Vrací: "object"
Array.isArray([]); // Vrací: true
typeof {}; // Vrací: "object"
Array.isArray({}); // Vrací: false
```

C. Operátory

- **Aritmetické operátory:**
 - Sčítání (+), odčítání (-), násobení (*), dělení (/), modulo (%).
- **Porovnávací operátory:**
 - Rovná se (==), striktní rovná se (===), nerovná se (!=), striktní nerovná se (!==), větší než (>), menší než (<), větší nebo rovno (>=), menší nebo rovno (<=).
- **Logické operátory:**
 - A (&&), nebo (||), negace (!).
- **Přiřazovací operátory:**
 - Jednoduché přiřazení (=), součtové přiřazení (+=), rozdílové přiřazení (-=) atd.
- **Inkrementní/dekrementní operátory:**
 - ++ , -- .

D. Základní syntaxe a konvence

- **Psaní a spouštění JS skriptů:**
 - Struktura JS souboru.
 - Výstup pomocí `console.log`.
- **Komentáře:**
 - Jednořádkové (//) a víceřádkové (/* */) komentáře.
- **Odsazení a čitelnost kódu:**
 - Důležitost konzistentního odsazení.
 - Konvence pojmenování a best practices pro čistý kód.

E. Základy ladění

- **Použití `console.log` k ladění:**
 - Jak vypisovat proměnné a výrazy.
 - Kdy běžně používat `console.log`.
- **Úvod do vývojářských nástrojů prohlížeče:**

- Přístup ke konzoli.
- Základní ladicí techniky — breakpoints a sledování výrazů.

Co je console.log a jak ho využít při ladění?

Definice:

`console.log` je funkce v JavaScriptu, která vypíše informace do webové konzole. Běžně je používána k ladění — zobrazuje hodnoty proměnných, zprávy nebo průběh vykonávání kódu.

Podrobné vysvětlení:

- **Základní použití:**

- **Výpis hodnoty:**

```
let greeting = "Hello, World!";
console.log(greeting); // Vypíše: Hello, World!
```

- **Výpis vícero hodnot:**

```
let user = "Bob";
let score = 85;
console.log("Uživatel:", user, "Skóre:", score); // Uživatel: Bob Skóre: 85
```

- **Logování objektů a polí:**

```
let user = { name: "Charlie", age: 25 };
console.log(user); // Vypíše objekt user
```

- **Ladění aplikací:**

- **Sledování průběhu:**

```
function add(a, b) {
  console.log("Sčítám:", a, b);
  return a + b;
}
add(5, 3); // Sčítám: 5 3
```

- **Sledování stavů proměnných:**

```

let total = 0;
for (let i = 1; i <= 5; i++) {
    total += i;
    console.log("Po přidání", i, "je total", total);
}
// Po přidání 1 je total 1
// Po přidání 2 je total 3
// Po přidání 3 je total 6
// Po přidání 4 je total 10
// Po přidání 5 je total 15

```

- **Použití s vývojářskými nástroji:**

- **Přístup ke konzoli:**

- Otevřete vývojářské nástroje prohlížeče (obvykle klávesou F12 nebo Ctrl+Shift+I).
 - Přepněte na záložku "Console" a zobrazte výstupy z `console.log`.

- **Best practices:**

- **Srozumitelné zprávy:**

- Zprávy použijte popisné.

```

console.log("Pokus o přihlášení uživatele:", username);

```

- **Odstraňte nebo zakomentujte logy v produkci:**

- Nadměrné logování může konzoli zahlcovat či odhalit citlivé informace.

- **Pokročilé logování:**

- **Seskupování logů:**

```

console.group("Detaily uživatele");
console.log("Jméno:", user.name);
console.log("Věk:", user.age);
console.groupEnd();

```

- **Styling logů:**

```

console.log("%cToto je stylovaná zpráva", "color: blue; font-size: 16px;");

```

F. Praktické příklady

- **Jednoduché programy v JavaScriptu:**

- Vytváření a manipulace s proměnnými.

- Provedení aritmetických a logických operací.
- Psaní funkcí pro opakující se úkoly.
- **Reálné scénáře:**
 - Ukázky použití těchto základů JavaScriptu při psaní testů v Cypressu.

2. Praktické aktivity: cvičení a návrhy webových funkcí

A. Cvičení na deklaraci proměnných a scope

- **Cvičení:**
 - Deklarujte proměnné pomocí `var`, `let` a `const`.
 - Srovnejte a pozorujte jejich scope v různých blocích (např. v cyklech nebo podmínkách).
- **Návrh webové funkce:**
 - Vytvořte jednoduchou HTML stránku s JavaScriptem, která deklaruje proměnné v různých scope a vypisuje jejich hodnoty do konzole pro pochopení chování scope.

B. Cvičení na datové typy a přetypování

- **Cvičení:**
 - Přidejte proměnným různé datové typy a pomocí `typeof` je identifikujte.
 - Proveďte operace ilustrující přetypování (např. sčítání řetězce a čísla).
- **Návrh webové funkce:**
 - Vyvořte malý skript, který přijímá vstup uživatele z formuláře a zpracuje jej pomocí různých datových typů, ukazujíc přetypování a kontrolu typů.

C. Cvičení na operátory

- **Cvičení:**
 - Pište výrazy s použitím různých aritmetických, porovnávacích, logických a přiřazovacích operátorů.
 - Předpovězte a ověřte výsledky pomocí `console.log`.
- **Návrh webové funkce:**
 - Implementujte základní kalkulačku na HTML stránce, která používá JavaScript k provádění operací podle vstupů uživatele s využitím různých operátorů.

D. Psaní a spouštění jednoduchých skriptů

- **Cvičení:**
 - Vytvořte JS soubor s deklarací proměnných, operacemi a funkcemi.

- Spusťte skript pomocí Node.js a sledujte výstup v konzoli.
- **Návrh webové funkce:**
 - Vyvořte jednoduchou to-do aplikaci, kde uživatelé mohou přidávat a odebírat úkoly, procvičujte manipulaci s proměnnými a základní interakci s DOM.

E. Ladění pomocí `console.log` a vývojářských nástrojů

- **Cvičení:**
 - Vkládejte `console.log` příkazy do různých částí kódu pro sledování hodnot a průběhu.
 - Nastavujte breakpointy a sledujte proměnné během spuštění pomocí vývojářských nástrojů prohlížeče.
- **Návrh webové funkce:**
 - Vylepšete svou to-do aplikaci vložением ladicích výpisů ke sledování uživatelských akcí a změn dat.

3. Možné otázky studentů

A. Obecné dotazy na JavaScript:

1. **Jaký je rozdíl mezi `var`, `let` a `const` ?**
 - Pochopení scope a možnosti přeložení.
2. **Proč je JavaScript důležitý pro automatizaci testování s Cypress?**
 - Jeho role při psaní testovacích skriptů a interakci s webovými aplikacemi.
3. **Mohu používat JavaScript pro složitější operace v Cypress testech?**
 - Ano, JavaScript umožňuje flexibilní a silné skriptování testů.

B. Proměnné a datové typy:

1. **Proč bych měl používat `let` nebo `const` místo `var` ?**
 - Vyvarujte se problémům se scope a zvyšte spolehlivost kódu.
2. **Jak ovlivňuje přetypování moje testovací skripty?**
 - Může vést k nečekaným výsledkům, pokud není správně zvládnuto.

C. Operátory:

1. **Kdy mám použít `===` místo `==` ?**
 - Aby se porovnávala hodnota i typ, což zabrání problémům s přetypováním.
2. **Jak efektivně využít logické operátory v testovacích podmínkách?**
 - Pro kombinaci více podmínek pro komplexnější asserty.

D. Základní syntaxe a konvence:

1. Jaké jsou nejlepší postupy pojmenování proměnných v JavaScriptu?
 - Používat popisná a smysluplná jména, dodržovat camelCase.
2. Jak důležitá je čitelnost kódu v test automatizaci?
 - Je klíčová pro údržbu a škálování testovacích sad.

E. Základy ladění:

1. Jaké jsou běžné ladicí techniky v JavaScriptu?
 - Používání `console.log`, breakpointů a sledování výrazů.
2. Jak efektivně využívat vývojářské nástroje pro ladění testů?
 - Inspekci elementů, sledováním konzole a krokováním kódu.

F. Praktické aplikace:

1. Můžete ukázat příklad použití funkcí v Cypress testech?
 - Funkce mohou rekapitulovat opakující se akce, což činí testy modulárními a udržitelnými.
2. Jak souvisí pole a objekty v JavaScriptu se správou testovacích dat v Cypress?
 - Lze je použít pro efektivní uchovávání a manipulaci s testovacími daty.

4. Doplnkové materiály: doporučení

A. Oficiální dokumentace a průvodci:

- Základy JavaScriptu:
 - [MDN JavaScript Guide](#)
- Dokumentace k Cypress:
 - [Cypress Getting Started](#)
- Node.js Dokumentace:
 - [Node.js Official Docs](#)

B. Interaktivní tutoriály a kurzy:

- Codecademy:
 - [Learn JavaScript](#)
- FreeCodeCamp:
 - [JavaScript Algorithms and Data Structures](#)
- [JavaScript.info](#):

- [The Modern JavaScript Tutorial](#)

C. Video tutoriály:

- **Traversy Media:**
 - [JavaScript Crash Course for Beginners](#)
- **Academind:**
 - [JavaScript Basics](#)
- **The Net Ninja:**
 - [JavaScript Tutorial for Beginners](#)

D. Platformy na procvičování:

- **LeetCode:**
 - [JavaScript Challenges](#)
- **HackerRank:**
 - [JavaScript Practice](#)
- **Exercism:**
 - [JavaScript Track](#)

E. Komunita a podpora:

- **Stack Overflow:**
 - [JavaScript Tag](#)
- **Reddit:**
 - [r/javascript](#)
- **Discord komunity:**
 - Připojte se do Discord serverů zaměřených na JavaScript nebo Cypress pro okamžitou pomoc a diskusi.

F. Další čtení:

- **Knihy:**
 - *Eloquent JavaScript* od Marijna Haverbekea ([Online verze](#))
 - *JavaScript: The Good Parts* od Douglase Crockforda
- **Články:**
 - [JavaScript Basics](#) od W3Schools
 - [Understanding JavaScript Variables](#) od FreeCodeCamp

5. Návrh rozvržení lekce na 3 hodiny

1. hodina: Úvod do JavaScriptu (60 minut)

- **Co je JavaScript? (15 minut):**
 - Přehled JavaScriptu a jeho významu ve vývoji webu.
 - Důležitost v automatizaci testů pomocí Cypressu.
- **JavaScript v automatizaci testů (15 minut):**
 - Jak JavaScript spolupracuje s Cypress.
 - Výhody použití JavaScriptu pro psaní testů.
- **Proměnné a datové typy (30 minut):**
 - Vysvětlení `var`, `let`, `const`.
 - Přehled různých datových typů.
 - Příklady a rychlé ukázky.

2. hodina: Operátory a základní syntaxe (60 minut)

- **Operátory (30 minut):**
 - Detailně o aritmetických, porovnávacích, logických a přiřazovacích operátorech.
 - Praktické příklady použití různých operátorů.
- **Základní syntaxe a kodovací konvence (20 minut):**
 - Psaní čistého a čitelného JS kódu.
 - Význam komentářů a správného odsazení.
- **Základy ladění (10 minut):**
 - Použití `console.log` k ladění.
 - Úvod do vývojářských nástrojů prohlížeče.

3. hodina: Praktická cvičení a Q&A (60 minut)

- **Praktická cvičení (40 minut):**
 - **Deklarace proměnných a scope:**
 - Studenti deklarují proměnné pomocí `var`, `let`, `const` a sledují rozdíly ve scope.
 - **Datové typy a přetypování:**
 - Přidělují různé typy proměnným a provádějí operace demonstrující přetypování.
 - **Procvičení operátorů:**
 - Píší výrazy s různými operátory a předpovídají výsledky.
- **Q&A část (20 minut):**
 - Odpovídání na studentské dotazy.
 - Vyjasnění nejasností a upevnění klíčových konceptů lekce.

6. Další doporučení

A. Interaktivní ukázky:

- **Live kódování:**
 - Ukázat deklaraci proměnných, operace a definici funkcí v reálném čase.
 - Ukažte, jak změny proměnných ovlivňují výstupy přes `console.log`.
- **Ukázka ladění:**
 - Projděte jednoduchý skript s použitím `console.log` k ladění.
 - Seznamte studenty se základními funkcemi vývojářských nástrojů prohlížeče.

B. Zapojení vizuálních prvků:

- **Prezentace s příklady kódu:**
 - Používejte jasné a stručné ukázky kódu k vysvětlení konceptů.
- **Diagramy:**
 - Vizuální znázornění scope proměnných a datových typů.
- **Živé příklady:**
 - Ukažte reálné scénáře, kde základní koncepty JavaScriptu platí pro testování v Cypress.

C. Podporujte zapojení:

- **Párové programování:**
 - Studenti mohou pracovat ve dvojicích a řešit úkoly společně.
- **Rychlé ankety a kvízy:**
 - Používejte rychlé ankety/kvízy pro ověření porozumění a zvýšení zapojení.

D. Dbejte na jasné návody:

- **Krok za krokem:**
 - Nabídněte detailní návody k praktickým aktivitám, aby studenti drželi krok.
- **Tipy pro řešení problémů:**
 - Připravte řešení pro běžné chyby (např. syntax) a návody na jejich odstranění.

E. Podporujte přátelské prostředí:

- **Podporujte dotazy:**
 - Vytvářejte otevřenou atmosféru pro otázky a žádosti o pomoc.
- **Poskytujte příklady:**

- Nabídněte více příkladů ke každému konceptu pro různé styly učení.